

run-tests no status	npm@latest v10.1.0	license MIT	URLPattern polyfills URLPattern is a new web API for matching URLs. Its intended to both provide a convenient API for web developers and to be usable in other web APIs that need to match URLs; e.g. service workers. The explainer discusses the motivating use cases. This is a polyfill for the URLPattern API so that the feature is available in browsers that
1 8 might not be available, and before you load the polyfill it environments because exist is needed in some 'urlpattern' does not ignore: Property The line with // @ts- Note: globalThis.urlpattern = URLPattern import {URLPattern} from "urlpattern-polyfill";	2 7 /* In case you want to replace an existing implementation with the polyfill: */ import "urlpattern-polyfill"; /* already polyfilled. runtime or natively supported by the * loads it on demand, thus when not	3 9 /* The above is the recommended way to load the ESM module, as it only loading as ESM module	4 5 The polyfill works in browsers (ESM module) and in Node.js either via import (ESM module) or via require (CJS module). The polyfill will only be loaded if the URLPattern doesn't already exist on the global object, and in that case it will add it to the global object. How to load the polyfill don't support it natively. This polyfill passes the same web platform test suite.
the feature-check in the if statement gives an TypeScript error. The whole idea is that it loads when its not there. loading as CommonJs module // Conditional CJS module loading (Node.js) if (!globalThis.URLPattern) { require("urlpattern-polyfill"); }	/* The above is the recommended way to load the CommonJs module, as it only * loads it on demand, thus when not natively supported by the runtime or already polyfilled. */ require("urlpattern-polyfill"); /*	* In case you want to replace an existing implementation with the polyfill: const {URLPattern} = require("urlpattern-polyfill"); globalThis.URLPattern = URLPattern Note: No matter how you load the polyfill, when there is no	implementation in your environment, it will <i>always</i> add it to the global object. Basic example let p = new URLPattern({ pathname: ' /foo/:name' }); let r = p.exec('https://example.com/foo/bar'); console.log(r.pathname.input); // " /foo/bar"
9 16 The pattern in this case can be made simpler without the origin check by leaving off the baseURL.	10 15 for (let url in url_list) { const r = p.exec(url); // skip non-matches if (!r) { continue; } if (r.pathname.groups['filetype'] === 'jpg') { // process jpg	11 14 Match same-origin jpg or png URLs. // Note: This uses a named group to make it easier to access the result later. const p = new URLPattern({ pathname: '/*.:filetype(jpg png)', baseURL: self.location }); Example of matching same-origin JPG or PNG requests	12 13 let r2 = p.exec({ pathname: '/foo/baz' }); console.log(r2.pathname.groups.name); // "baz" console.log(r.pathname.groups.name); // "bar"

<pre>// Match any URL ending with 'jpg' or 'png'. const p = new URLPattern({ pathname: '/*.:filetype(jpg png)' });</pre> Example of Short Form Support We are planning to also support a “short form” for initializing URLPattern objects. This is supported by the polyfill but not yet by the Chromium implementation. For example: 17	<pre>const p = new URLPattern("https:// *.example.com/foo/*");</pre> Or: <pre>const p = new URLPattern("foo/*", self.location);</pre> 18	API reference API overview with typeScript type annotations is found below. Associated browser Web IDL can be found here. <pre>type URLPatternInput = URLPatternInit string;</pre> <pre>class URLPattern {</pre> 19	<pre>constructor(init?: URLPatternInput, baseURL?: string);</pre> <pre>test(input?: URLPatternInput, baseURL?: string): boolean;</pre> <pre>exec(input?: URLPatternInput, baseURL?: string): URLPatternResult null;</pre> <pre>readonly protocol: string; readonly username: string;</pre> 20
24 <pre> { [key: string]: string undefined; } };</pre>	<pre>interface URLPatternComponentResult { groups: { input: string; } hash: URLPatternComponentResult; search: URLPatternComponentResult; pathname: URLPatternComponentResult; port: URLPatternComponentResult; hostname: URLPatternComponentResult; password: URLPatternComponentResult;</pre> 23	<pre>interface URLPatternResult { protocol?: string; hostname?: string; port?: string; pathname?: string; search?: string; hash?: string; }</pre> <pre>username: URLPatternComponentResult; password: URLPatternComponentResult;</pre> 22	<pre>interface URLPatternInit { readonly password: string; readonly hostname: string; readonly port: string; readonly pathname: string; readonly search: string; readonly hash: string; }</pre> 21
Pattern syntax The pattern syntax here is based on what is used in the popular path-to-regexp library. • An understanding of a “divider” that separates segments of the string. For the pathname this is typically the "/" character. • A regex group defined by an enclosed set of parentheses. Inside of the parentheses a general regex may be defined. 25	• A named group that matches characters until the next divider. The named group begins with a ":" character and then a name. For example, "/:foo/:bar" has two named groups. • A custom regex for a named group. In this case a set of parentheses with a regex immediately follows the named group; e.g. "/:foo.(*)" will override the default of matching to the next divider. • A modifier may optionally follow a regex or named group. A modifier is a "?", "*", or "+" 26	functions just as they do in regular expressions. When a group is optional or repeated and it's preceded by a divider then the divider is also optional or repeated. For example, "/foo/:bar?" will match "/foo", "/foo/", or "/foo/baz". Escaping the divider will make it required instead. • A way to greedily match characters, even across dividers, by using "(.*)" (so-called unnamed groups). 27	Currently we plan to have these known differences with path-to-regexp: • No support for custom prefixes and suffixes. 28
Breaking changes • V9.0.0 drops support for NodeJS 14 and lower. NodeJS 15 or higher is required. This is due to using private class fields, so we can have better optimizations. There is <i>No</i> change in functionality, but we were able to reduce the size of the polyfill by ~2.5KB (~13%), thanks to a pr #118 from @jimmywaring. 32	<pre>new URL("https:// 01erlakkernaðdetermit.dk").hostname // "xn--lerlakkernrðdetermit-dub078a.dk"</pre> 31	a developer would need to URL encode unicode characters before passing the pattern into the constructor. Similarly, the constructor does not do things like flattening pathnames such as /foo/./bar to /bar. Currently the pattern must be written to target canonical URL output manually. It does, however, perform these operations for test() and exec() input. Encoding components can easily be done manually, but do not encoding the pattern syntax: 30	URLs have a canonical form that is based on ASCII, meaning that internationalized domain names (hostnames) also have a canonical ASCII based representation, and that other components such as hash, search and pathname are encoded using percent encoding. Currently URLPattern does not perform any encoding or normalization of the patterns. So 29

Learn more • Explainer • Design Document 33	Reporting a security issue If you have information about a security issue or vulnerability with an Intel-maintained open source project on https://github.com/intel, please send an e-mail to secure@intel.com. Encrypt sensitive information using our PGP public key. For issues related to Intel products, please visit https://security-center.intel.com. 34	35	36
40	39	38	37
41	42	43	44
48	47	46	45